

Patristic Library Assistant

Programmer-Facing Feature Roadmap and Engineering Specification

Detailed implementation guidance for the next moat-building features. This version is written for a full-stack engineer, technical co-founder, or contract developer.

Document purpose Translate the strategic roadmap into implementation-ready feature definitions, service boundaries, schema additions, API patterns, UI surfaces, and rollout sequencing.	Assumed stack FastAPI, Next.js, PostgreSQL, Qdrant, Redis, Stripe, Claude API, optional graph layer, optional local deployment path.
--	--

1. Executive implementation summary

The current product definition already covers closed-corpus retrieval, ingestion, approval workflows, confidence-tier logic, liturgical context, graph readiness, multi-tenancy, and a future tenant network. The next wave should not add generic chatbot features. It should build **trust infrastructure**, **governance infrastructure**, **scholarly workflow infrastructure**, and **corpus-improvement infrastructure**.

- **Trust infrastructure**: visible provenance, quote verification, source attestation, textual confidence.
- **Governance infrastructure**: hierarchical policy inheritance across bishop, diocese, seminary, parish, or monastery contexts.
- **Scholarly workflow infrastructure**: alignment, witness graphs, thesis-grade exports, dispute views.
- **Corpus-improvement infrastructure**: turning unanswered questions into structured content production and ingestion loops.

Recommended build order

Priority	Feature	Moat Strength	Engineering Effort	Main Buyer
1	Source Attestation Layer	Very High	Medium	Seminaries / dioceses / monasteries
2	Patristic Citation Resolver	High	Medium	Scholars / creators / clergy
3	Corpus Gap-to-Content Loop	Very High	Medium	Ministries / creators / parishes
4	Synodal Governance Engine	Very High	High	Dioceses / patriarchates
5	Scholarly Workbench Mode	High	Medium	Seminaries / researchers
6	Epistemic Answer Modes	High	Low-Medium	All serious institutional buyers
7	Parallel Text Alignment Engine	Very High	High	Seminaries / translators
8	Manuscript Witness Graph	Extreme	Very High	Monasteries / archives / academia

Priority	Feature	Moat Strength	Engineering Effort	Main Buyer
9	Local-First Review Console	High	Medium	Monasteries / privacy-sensitive institutions
10	Agentic Institutional Workflows	Very High	High	Dioceses / seminaries / ministries

2. Source Attestation Layer

Every answer should expose not just a citation but a complete chain of custody: where the passage came from, how it was extracted, who approved it, whether the quote is exact or normalized, and whether it has parallel witnesses elsewhere in the corpus. This feature converts internal approval metadata into a visible trust product.

Primary moat: Trust infrastructure and procurement-grade auditability.

Why this matters

- Institutional buyers need more than a source title; they need to know whether a passage is clergy-approved, machine-transcribed, manually corrected, or translated.
- The current architecture already stores approved chunks and audit events; this feature formalizes that metadata and surfaces it in retrieval and UI.
- It strengthens theological safety because developers can now separate *retrieval confidence* from *source integrity confidence*.

Core engineering components

- Attestation metadata service attached to ingestion and approval flows.
- Provenance object generated per cited passage in every answer payload.
- UI panel showing integrity status, approval status, and derivation path.
- Validation rules to block unsupported quote labels such as 'exact quote' when the passage is actually normalized or translated.

Suggested schema additions

Table: source_attestations(id, source_id, source_hash, extraction_method, transcription_method, transcription_confidence, translation_status, original_language, created_at)

Table: chunk_attestations(chunk_id, attestation_id, quote_integrity ENUM['verbatim', 'normalized', 'translated', 'summary'], approved_by, approved_at, manual_review_required BOOL, review_notes TEXT)

JSON field on answer citations: {chunk_id, quote_integrity, approval_status, approved_by_name, source_hash, derivation_path, parallel_witness_count}

API / service surfaces

- POST /api/v1/ingest -> create source_attestation record after extraction
- PATCH /api/v1/corpus/{chunk_id}/approve -> attach approver and approval timestamp
- GET /api/v1/attestation/source/{source_id} -> detailed audit object for admin UI
- GET /api/v1/query response -> include attestation sub-object per citation
- Background job: recompute source_hash and detect file drift after source replacement

Processing flow

- 1. Ingestion extracts file text, transcript text, or OCR text.
- 2. System records extraction_method and source fingerprint.
- 3. Reviewer approves chunks or flags them for manual verification.
- 4. Query response builder merges retrieval result with attestation metadata.
- 5. UI shows badges such as Approved, OCR-derived, Human-corrected, or Translation-backed.

Key edge cases

- A source file is replaced after approval; chunk pointers remain valid but attestation must show superseded status.
- Machine transcript confidence is low; response should still cite it but label it clearly.
- A translated quote appears in English while original Greek exists elsewhere; the UI must distinguish translation from original-language quotation.
- Bulk import creates thousands of chunks with no manual review; developer should support staged trust levels.

Implementation order

- Phase A: schema additions and hidden metadata capture.
- Phase B: admin attestation viewer and response payload enrichment.
- Phase C: user-facing trust badges and integrity filters.
- Phase D: export attestation reports for seminary or diocesan procurement.

3. Patristic Citation Resolver

Orthodox content frequently contains paraphrases, loose attributions, sermon summaries, or quotes passed through modern teachers. The resolver classifies each citation as exact, close paraphrase, likely allusion, misattributed, or unresolved. Over time, this becomes a corpus-cleaning engine and a proprietary scholarly dataset.

Primary moat: Data quality moat and citation integrity moat.

Why this matters

- Seminaries and creators need to distinguish exact patristic citation from modern paraphrase.
- It reduces reputational risk by detecting fabricated or weakly sourced patristic quotes.
- It gives the platform a structured advantage over systems that only retrieve text similarity.

Core engineering components

- Canonical-source search index over trusted patristic editions and translations.
- Resolver pipeline combining fuzzy string search, vector similarity, and metadata-aware rank fusion.
- Admin review queue for unresolved or contested matches.
- Citation-classification cache so repeated ingestion does not re-run expensive search steps.

Suggested schema additions

Table: citation_candidates(id, source_chunk_id, extracted_quote, cited_father, cited_work, cited_location_text, normalized_quote)

```
Table: citation_matches(candidate_id, canonical_chunk_id, lexical_score, embedding_score,
combined_score, classification ENUM['exact', 'paraphrase', 'allusion', 'misattributed', 'unresolved'],
reviewer_id, reviewer_note)
```

```
Materialized view or cache: citation_resolution_summary(source_id, exact_count, unresolved_count,
misattributed_count)
```

API / service surfaces

- Background worker: resolve citations for newly ingested sources
- GET /api/v1/citations/source/{source_id} -> list all extracted citations and statuses
- PATCH /api/v1/citations/{candidate_id} -> reviewer override for classification
- Query API can optionally expose citation health in answer metadata for modern lecture sources

Processing flow

- 1. During ingestion, extract candidate quote spans and named references from transcript or text.
- 2. Normalize punctuation, orthography, and spelling variants.
- 3. Search canonical corpus using both lexical and semantic methods.
- 4. Rank matches and assign provisional classification thresholds.
- 5. Route borderline cases to review queue.
- 6. Store result and expose citation health in source browser.

Key edge cases

- A modern lecturer paraphrases Chrysostom without marking it as quotation.
- The same patristic text exists in multiple translations with different wording.
- Greek original and English translation both match partially; developer must decide whether to privilege language-aligned corpora.
- A candidate match is semantically similar but not actually the quoted source.

Implementation order

- Start with English-only canonical matching over the starter corpus.
- Then add Greek-original support and translation linkage.
- Then add reviewer overrides and confidence analytics.
- Finally expose citation integrity scoring in public-facing output.

4. Corpus Gap-to-Content Production Loop

The current product already records RED-tier and low-confidence questions. This feature turns them into a demand-sensing editorial engine: cluster unanswered questions, rank them by business value, map them to likely primary sources, and generate a structured production brief for a priest, content creator, or researcher.

Primary moat: Proprietary data generation and self-improving corpus moat.

Why this matters

- It converts user demand into a content roadmap rather than leaving flagged queries as passive analytics.

- For creators and ministries, it directly informs what lecture, article, or Q&A; should be recorded next.
- For seminaries or dioceses, it reveals persistent catechetical blind spots.

Core engineering components

- Semantic clustering service over flagged queries.
- Priority scoring model based on frequency, recency, user segment, revenue tier, and theological sensitivity.
- Brief generator that proposes sources, Fathers, councils, and related existing content.
- Closed loop from published content back into ingestion and gap-resolution tracking.

Suggested schema additions

Table: flagged_query_events(id, tenant_id, query_text, normalized_query, tier, user_segment, created_at)

Table: query_gap_clusters(id, tenant_id, title, centroid_embedding, frequency, urgency_score, last_seen_at, status ENUM['open', 'planned', 'recorded', 'ingested', 'resolved'])

Table: content_briefs(id, cluster_id, draft_title, suggested_sources JSONB, suggested_fathers JSONB, suggested_format ENUM['lecture', 'article', 'study_packet', 'faq'], assigned_to, due_date)

API / service surfaces

- GET /api/v1/flagged/clusters -> ranked list of unresolved query clusters
- POST /api/v1/briefs/generate/{cluster_id} -> generate a content brief
- PATCH /api/v1/briefs/{brief_id} -> assign owner or mark as produced
- Webhook or manual action linking new content source_id back to cluster_id for closure analytics

Processing flow

- 1. Capture every RED or weak-YELLOW query event.
- 2. Normalize and embed query text.
- 3. Run periodic clustering job per tenant.
- 4. Rank clusters by urgency score.
- 5. Generate editorial brief with recommended patristic and local sources.
- 6. When new source is ingested, link it to the cluster and evaluate whether coverage improved.

Key edge cases

- High-frequency but low-value queries such as basic navigation questions should not dominate cluster priority.
- Sensitive moral or pastoral topics may need human review before automated brief generation.
- Different tenants may need isolated clustering even if questions are semantically similar.
- A single new lecture may partially address several clusters; closure logic should support many-to-many relationships.

Implementation order

- Phase A: query normalization and embedding storage.
- Phase B: per-tenant clustering and ranking dashboard.
- Phase C: auto-generated content briefs and assignment flow.

- Phase D: coverage-delta measurement after re-ingestion.

5. Synodal Governance Engine

The platform needs a rule system that models ecclesial authority structure rather than flat document retrieval. A diocesan tenant may need bishop-approved baseline guidance, seminary-level corpus policies, and parish-level additions that are valid locally but must not override higher-level constraints without explicit authorization.

Primary moat: Institutional lock-in and ecclesial governance moat.

Why this matters

- Generic RAG systems have no concept of hierarchical theological authority.
- This feature is decisive for dioceses, patriarchates, and seminaries with formal review structures.
- It supports controlled pluralism: local material can exist without silently overruling approved institutional position.

Core engineering components

- Policy inheritance engine with override rules.
- Conflict detector comparing local chunks and policies against higher-level canonical or institutional baselines.
- Answer composer capable of separating patristic evidence from official institutional stance.
- Mode switch controlling whether a user sees open scholarly material, institutionally approved position, or both.

Suggested schema additions

Table: policy_nodes(id, tenant_id, parent_node_id, node_type ENUM['global', 'jurisdiction', 'diocese', 'seminary', 'parish', 'monastery'], name)
Table: policy_rules(id, policy_node_id, rule_key, rule_value JSONB, precedence, active_from, active_to)
Table: corpus_scope_rules(id, policy_node_id, source_id, visibility_mode ENUM['allow', 'suppress', 'scholarly_only', 'admin_only'])
Table: conflict_events(id, chunk_id, conflicting_rule_id, severity, detected_at, resolved_by)

API / service surfaces

- GET /api/v1/policies/tree -> visualize inheritance graph
- PATCH /api/v1/policies/rule/{id} -> modify governance rule
- POST /api/v1/policies/evaluate -> simulate answer policy resolution for a sample query
- Query service must accept user_mode or answer_mode to determine which policies apply

Processing flow

- 1. Build policy tree for tenant or multi-site deployment.
- 2. At query time, retrieve candidate chunks as usual.
- 3. Apply policy filters and precedence rules before final prompt assembly.
- 4. If conflicts exist, either suppress conflicting chunks or surface them under scholarly mode.
- 5. Emit governance explanation metadata with answer.

Key edge cases

- Parish source conflicts with diocesan instruction but is historically important; should be `scholarly_only` rather than deleted.
- A policy changes over time; answers for past exports may need reproducible historical context.
- A user has member role while admin previews scholarly mode; access control must be explicit.
- Cross-tenant discovery later must not violate policy visibility rules.

Implementation order

- Implement policy tree and simulation endpoint first.
- Then apply rules to corpus visibility and answer composition.
- Then add conflict analytics and scholarly-mode branching.
- Then support multi-site diocesan inheritance at enterprise tier.

6. Scholarly Workbench Mode

This mode is not just a better answer UI. It is a research environment for assembling evidence, inspecting doctrinal development, comparing witnesses, exporting argument bundles, and deliberately handling unresolved tensions instead of flattening everything into a single chat reply.

Primary moat: Workflow lock-in for advanced users.

Why this matters

- Seminary users need thesis-grade research support rather than only conversational answers.
- It creates daily workflow dependency for students, professors, and translators.
- It differentiates the platform on workflow even where a competitor has more raw documents.

Core engineering components

- Saved research workspace with folders, evidence bundles, and note objects.
- Claim extraction engine producing atomic claims linked to supporting or opposing passages.
- Timeline / doctrinal development visualization using source dates and councils.
- Export module for bibliography, footnotes, and research packets.

Suggested schema additions

Table: `workspaces(id, tenant_id, owner_user_id, title, description)`

Table: `workspace_items(id, workspace_id, item_type
ENUM['query', 'claim', 'source', 'note', 'bundle', 'timeline_event'], ref_id, metadata JSONB)`

Table: `claims(id, workspace_id, claim_text, claim_type
ENUM['descriptive', 'doctrinal', 'historical'], status ENUM['supported', 'contested', 'insufficient'])`

API / service surfaces

- POST `/api/v1/workspaces` -> create research workspace
- POST `/api/v1/workspaces/{id}/claims/extract` -> derive claims from selected passages

- POST /api/v1/workspaces/{id}/bundle -> build evidence packet
- GET /api/v1/workspaces/{id}/export?format=pdf|docx|json

Processing flow

- 1. User runs a query in scholarly mode.
- 2. User saves answer passages into a workspace.
- 3. System extracts claims and groups supporting and opposing citations.
- 4. Timeline view orders sources chronologically.
- 5. User exports packet with citations, notes, and bibliography.

Key edge cases

- Not every source has clean dating information.
- Different translations may collapse distinct claims into similar English wording.
- The system should support insufficient-evidence outcomes without forcing classification.
- Workspace export must preserve citation traceability back to approved chunks.

Implementation order

- Start with saved bundles and manual note-taking.
- Add claim extraction and timeline view next.
- Then add advanced exports and dispute mapping.
- Finally add collaborative professor-student review if needed.

7. Epistemic Answer Modes

The product should explicitly distinguish between different kinds of theological output rather than using one generic answer form. A user may want direct quotation, consensus synthesis, historical development, institutional policy, or disputed scholarly framing. Each mode has different retrieval, ranking, and prompt composition rules.

Primary moat: Control-layer moat and explainability moat.

Why this matters

- It prevents category errors such as giving a harmonized answer when the user actually needs source divergence.
- It makes system behavior legible to institutional reviewers.
- It also provides clearer QA and test coverage because each answer mode has explicit acceptance criteria.

Core engineering components

- Mode selector in UI and query API.
- Per-mode retrieval templates and prompt templates.
- Per-mode output schema with required fields.
- Per-mode evaluation suite in staging.

Suggested schema additions


```
Enum on queries and saved sessions: answer_mode = ['direct_citation', 'consensus', 'historical_development', 'institutional_policy', 'scholarly_dispute']
```

Mode-specific payload examples: consensus mode requires agreement_points, divergence_points, conciliar_resolution; direct_citation mode requires citation_spans and minimal_synthesis.

API / service surfaces

- POST /api/v1/query with answer_mode parameter
- GET /api/v1/query/modes -> document supported modes and response schemas
- Test harness endpoint or internal tool to validate each mode against canned prompts

Processing flow

1. User selects answer mode or defaults are inferred by product context.
2. Retrieval layer applies different ranking or filtering strategy.
3. Prompt composer injects mode-specific rules.
4. Post-processor validates required response sections.
5. UI renders specialized layout for that mode.

Key edge cases

- A follow-up question should preserve mode unless the user changes it.
- Historical mode may need weaker semantic proximity but stronger date coverage.
- Institutional policy mode must never silently omit that a policy is local rather than universal.
- Consensus mode must not fabricate agreement where evidence is sparse.

Implementation order

- Ship direct citation and consensus first.
- Then add institutional policy once governance engine exists.
- Then add historical and scholarly dispute modes with custom exports.

8. Parallel Text Alignment Engine

This engine aligns Greek original, English translation, alternate translations, and possibly lecture paraphrases back to the same patristic unit. It should support click-through from translated text to the underlying Greek, expose translation variance, and improve retrieval by linking equivalent passages across languages.

Primary moat: Scholarly and multilingual moat.

Why this matters

- Seminaries and translators need original-language visibility without leaving the platform.
- It strengthens multilingual expansion because language links become first-class data instead of UI decoration.
- It also improves citation resolver quality by distinguishing translation variance from citation inaccuracy.

Core engineering components

- Alignment pipeline at sentence or clause level.

- Canonical passage IDs that span original and translated variants.
- Viewer UI with synchronized panels or toggle.
- Fallback manual alignment tooling for low-confidence segments.

Suggested schema additions

Table: canonical_passages(id, work_id, passage_ref, language_primary, normalized_anchor_text)

Table: passage_alignments(id, canonical_passage_id, source_chunk_id, language_code, alignment_level ENUM['sentence', 'clause', 'paragraph'], confidence_score, offset_map JSONB)

Table: translation_variants(id, canonical_passage_id, source_chunk_id, variant_label, translator_name, notes)

API / service surfaces

- Alignment worker after ingestion of translations or originals
- GET /api/v1/passages/{canonical_id}/aligned -> return all language witnesses
- UI query option: show original / show parallel / show translation differences

Processing flow

- 1. Normalize source into comparable passage units.
- 2. Generate embeddings per unit and candidate link set.
- 3. Resolve alignments using lexical anchors and metadata such as work title and passage refs.
- 4. Persist offset mappings for synchronized highlighting.
- 5. Expose in citation and reading views.

Key edge cases

- One translation compresses two Greek sentences into one English sentence.
- Passage boundaries differ by edition.
- Lecture excerpts quote only fragments of the underlying text.
- Polytonic Greek and OCR noise may reduce lexical match quality.

Implementation order

- Start with manually curated starter corpus alignments.
- Then add automated candidate generation and reviewer UI.
- Then support cross-language retrieval boosts and synchronized reading.

9. Manuscript Witness Graph

If the product later enters manuscript digitization or archive partnerships, it should model not just documents but witness relationships: manuscript, edition, translation, excerpt tradition, and editorial dependency. This is the deepest long-term moat because the resulting structured dataset is hard to reproduce.

Primary moat: Archival data moat and long-term monopoly moat.

Why this matters

- It transforms the platform from 'AI over texts' to 'AI over tradition-bearing textual witnesses.'
- It becomes strategically important if Athonite, patriarchal, or seminary archival partnerships materialize.
- It enables more rigorous handling of textual variance, source lineage, and editorial confidence.

Core engineering components

- Graph store or graph-like relational layer.
- Witness entity model for manuscript, edition, translation, fragment, and modern citation witness.
- Variant and derivation edges.
- Viewer showing witness lineage and confidence.

Suggested schema additions

Nodes: witness(id, witness_type, title, date_range, language, repository, shelfmark, digital_asset_ref)

Edges: witness_edges(id, from_witness_id, to_witness_id, relation_type
ENUM['derives_from', 'variant_of', 'edition_of', 'translation_of', 'quotes', 'excerpted_in'],
confidence_score, note)

Table: witness_passage_links(witness_id, canonical_passage_id, location_ref, text_confidence)

API / service surfaces

- GET /api/v1/witness/{id} -> witness metadata and linked graph neighborhood
- GET /api/v1/passages/{id}/witnesses -> all witnesses for a canonical passage
- Admin import pipeline for archival metadata and OCR derivatives

Processing flow

- 1. Ingest edition or manuscript metadata.
- 2. Create witness node and attach repository identifiers.
- 3. Link witness to canonical passages and related witnesses.
- 4. Use graph neighborhood to enrich scholarly views and variant alerts.

Key edge cases

- A manuscript has uncertain dating or uncertain relationship to an edition.
- A digital scan exists but OCR confidence is too low for public retrieval.
- Not every witness should be queryable by end users; some may be metadata-only until reviewed.
- Graph edges should carry confidence scores rather than pretending certainty.

Implementation order

- Do not build this before attestation and alignment foundations exist.
- Begin with metadata graph even before full manuscript text ingestion.
- Then add passage links and variant support.
- Finally expose witness-aware scholarly outputs.

10. Local-First Review Console

Monasteries and privacy-sensitive institutions may accept review tooling before they accept cloud-based answer generation. A local-first console allows ingestion review, approval, attestation inspection, and deferred synchronization even before a fully offline inference stack is ready.

Primary moat: Privacy and sovereignty moat.

Why this matters

- It reduces trust friction for monasteries and archival custodians.
- It de-risks later offline deployment by separating review sovereignty from model sovereignty.
- It also provides a practical bridge for environments with intermittent connectivity.

Core engineering components

- Desktop or browser-based review client with local storage.
- Encrypted sync queue for metadata and approved artifacts.
- Conflict resolver for concurrent edits between local and cloud states.
- Package import/export for air-gapped review.

Suggested schema additions

Local SQLite or embedded Postgres for review state.

Table mirror subset: `local_sources`, `local_chunks`, `local_attestations`, `local_reviews`, `sync_events`.

Encryption metadata: `key_id`, `device_id`, `last_sync_checkpoint`.

API / service surfaces

- POST `/api/v1/sync/push` and `/sync/pull` with signed device credentials
- Export bundle command for air-gapped transfer
- Conflict resolution endpoint or client policy for last-writer / reviewer-priority behavior

Processing flow

- 1. Source package is imported locally.
- 2. Reviewer approves or annotates chunks.
- 3. Client stores encrypted changes in sync queue.
- 4. When connectivity exists, client pushes delta set.
- 5. Server validates signatures and merges state.

Key edge cases

- Two reviewers approve the same chunk differently on different devices.
- A source is deleted in cloud while still under local review.
- Air-gapped device cannot validate remote source replacement immediately.
- Key rotation must not orphan prior approvals.

Implementation order

- Phase A: local review-only cache with manual export/import.
- Phase B: authenticated sync protocol.
- Phase C: conflict resolution UI and device management.

11. Agentic Institutional Workflows

The strongest long-term workflow moat is not unrestricted AI agents. It is bounded, corpus-safe automation for institutional tasks: syllabus packets, bishop briefings, feast-day bundles, catechism lesson outlines, lecture-to-study-guide conversion, and parish FAQ drafting. The agent is valuable because it acts inside governance and provenance constraints.

Primary moat: Workflow dependency moat.

Why this matters

- This turns the product from a question-answering system into a work-producing system.
- It creates repeatable operational value for seminaries, dioceses, and ministries.
- It is defensible only if the underlying corpus, attestation, and policy layers are already strong.

Core engineering components

- Task-specific workflow agents with hard tool boundaries.
- Prompt templates and action plans per workflow.
- Artifact generation pipeline for PDF, DOCX, or web output.
- Human approval checkpoints before publication or sharing.

Suggested schema additions

Table: workflow_runs(id, tenant_id, workflow_type, initiated_by, status, inputs JSONB, outputs JSONB, approved_by, created_at)

Table: workflow_steps(id, run_id, step_type, status, started_at, finished_at, metadata JSONB)

Artifact table or file registry linking generated packets to workflow run and source citations

API / service surfaces

- POST /api/v1/workflows/run with workflow_type such as feast_packet, bishop_brief, syllabus_bundle, catechism_guide
- GET /api/v1/workflows/{id} -> status, step log, generated artifacts
- POST /api/v1/workflows/{id}/approve -> finalize artifact or send back for revision

Processing flow

- 1. User selects workflow and enters bounded inputs.
- 2. Orchestrator retrieves approved sources under policy constraints.
- 3. Agent performs decomposed steps such as query, cluster, summarize, organize, and export.
- 4. Post-processor verifies citation completeness and required sections.
- 5. Human reviewer approves final artifact.

Key edge cases

- Workflow should fail closed when coverage is insufficient rather than inventing content.
- Generated artifacts must embed source citations and, ideally, attestation status.
- Institutional policy mode must be enforced automatically for public-facing artifacts.
- Long workflows need resumable state and clear step logs for debugging.

Implementation order

- Start with one high-value workflow such as study packet or feast-day bundle.
- Then add bishop briefing and lecture-to-guide conversion.
- Only later build generalized orchestration once task templates are stable.

12. Cross-feature platform changes

Shared service layers

- A **metadata integrity layer** should sit between ingestion and retrieval so attestation, citation, alignment, and policy metadata are all first-class.
- A **background job framework** is required for citation resolution, query clustering, alignment, graph maintenance, and workflow orchestration.
- A **mode-aware prompt composer** should replace a single monolithic prompt builder.
- A **versioned export pipeline** should support PDF, DOCX, and machine-readable JSON artifacts.

Likely new infrastructure

- PostgreSQL remains primary system-of-record for governance, audit, workflow, and review data.
- Qdrant remains primary semantic retrieval layer.
- Optional graph layer can wait until witness graph or advanced lineage visualization becomes real product scope.
- A task queue such as Celery, RQ, Arq, or Dramatiq becomes increasingly important once background enrichment begins.
- File fingerprinting and artifact registry should be introduced early if attestation and export are core.

Test strategy

- Every feature should define **theological safety**, **data integrity**, and **explainability** acceptance criteria.
- Mode-specific canned queries should become part of CI or staging verification.
- Resolver and alignment systems need both automatic thresholds and manual reviewer overrides.
- Workflow agents should be tested with insufficient-coverage cases to ensure they fail safely.

13. Suggested 3-sprint sequence

Sprint	Primary scope	Deliverables
Sprint 1	Trust foundation	Attestation schema, approval metadata, citation payload enrichment, admin trust viewer

Sprint	Primary scope	Deliverables
Sprint 2	Corpus improvement loop	Flagged query clustering, content-brief generation, dashboard ranking, source-link closure flow
Sprint 3	Advanced control	Answer modes, policy simulation endpoint, first scholarly workspace bundle export

Bottom line: the strongest next features are the ones that create visible trust, formal ecclesial control, advanced scholarly workflow, and proprietary corpus-improvement data. Those are the features most likely to create a durable category lead over both a traditional research library and a generic AI-native RAG platform.